

Plato Listo

Alumno: Gabriel Alejandro Neculman Tranamil
Docente: Cristian Alejandro Iglesias Vera
Fecha: 24/11/2025



1. Introducción

En el presente informe se documenta detalladamente el proceso de implementación de métricas en un proyecto que busca modernizar y optimizar la gestión de pedidos en restaurantes.

Este proyecto consiste en un sistema digital diseñado para agilizar la toma y envío de órdenes. El funcionamiento es el siguiente: cuando el mesero atiende al cliente en la mesa y registra su pedido de manera electrónica (mediante un dispositivo móvil o una tablet), esta información se transmite instantáneamente a la cocina, donde se visualiza en otro dispositivo, como una pantalla o un computador.

El principal objetivo de esta herramienta es eliminar el proceso tradicional de anotar los pedidos en papel y tener que llevar físicamente la comanda a la cocina. Esto no solo reduce errores y tiempos de espera, sino que también permite que el mesero se enfoque más en la atención al cliente.

2. Objetivo del análisis

El objetivo principal de esta sección es demostrar que la aplicación es medible:

Comprobar la Captura de Datos: Analizar si el código de instrumentación (el que registra las métricas) está funcionando de forma eficiente. El objetivo es que los archivos que generan automáticamente y contengan los datos precisos requeridos (tiempos de sesión, registros de eventos, contadores).

Evaluar la Usabilidad Básica: Utilizar los datos generados para obtener conocimiento preliminar. Por ejemplo, si implementamos un contador, el análisis buscaría ¿Se usa más el botón de "Incrementar" o el de "Resetear"? ¿Cuánto tiempo promedio usa el usuario la aplicación antes de cerrarla? (Tiempo de sesión). Esto demuestra que la aplicación está lista para una fase de análisis de Experiencia de Usuario.

3. Métricas implementadas

Las métricas utilizadas e implementadas en el proyecto Plato Listo para documentar el proceso y la interacción del usuario son las siguientes:

- Inicio App: Registra el arranque de la aplicación.
- Duración de sesión: Mide el tiempo total que la aplicación estuvo activa (de inicio a cierre).
- Botón Mesero: Registra la pulsación para acceder a la pantalla del mesero.
- Botón Chef: Registra la pulsación para acceder a la pantalla del chef.
- Enviar Pedido: Registra el evento central del negocio, que es el envío exitoso de una orden a la cocina.

Explicación Técnica del Sistema de Instrumentación

La implementación se basa en un sistema de registro centralizado y estructurado, utilizando módulos estándar de Python

Módulos de Python Utilizados

Se utilizaron los siguientes módulos clave en main.py:

- logging: Módulo estándar de Python usado para centralizar el registro de eventos y la escritura en el archivo metrics.log.
- json: Fundamental para serializar cada entrada de métrica en formato JSON, asegurando que el log sea estructurado y fácil de parsear.
- datetime / time: Utilizados para generar *timestamps* en formato UTC ISO (datetime) y para el cálculo preciso de la duración de la sesión (time.time()).
- os: Empleado para construir la ruta al archivo metrics.log de manera segura y portable.

Registro Centralizado (record_event)

Se implementó la función central `record_event(name, details=None)` en la clase `PlatoListoApp`. Esta función encapsula toda la lógica de registro:

1. Genera un *timestamp* en UTC.
2. Construye un diccionario con las claves *timestamp*, *event* (el nombre de la métrica) y *details* (si se proporcionan, como en Enviar Pedido).
3. Serializa el diccionario a JSON (`json.dumps`).
4. Envía el mensaje JSON al *logger* configurado (`self.metrics_logger.info(msg)`).

Ventaja: Este punto centralizado permite mantener la lógica de UI separada de la lógica de *logging* y facilita el futuro reemplazo del `FileHandler` por un sistema de envío remoto (`HTTPHandler`) sin modificar las llamadas en la UI.

Implementación de Métricas de Ciclo de Vida y Eventos

Métricas de Ciclo de Vida (Inicio y Duración)

Se utilizaron los métodos de la clase `kivy.app` para registrar el tiempo de sesión:

- `on_start()`: Registra el evento Inicio App y almacena dos *timestamps*: uno para el registro (`datetime`) y otro para el cálculo de alta precisión (`time.time()`).
- `on_stop()`: Calcula la duración total de la sesión mediante la resta de los *timestamps* (`time.time() - self._start_time_ts`) y registra el evento Duración de Sesión, incluyendo la clave `duration_seconds` en los *details*.

Métricas de Interacción (Botones y Evento de Negocio)

- Registro de Botones (UI): La métrica de interacción más básica se implementó directamente en el archivo .kv. La llamada a `app.record_event()` se ejecuta en el *callback* `on_press` antes de la navegación, asegurando que el evento se capture.

Fragmento de código

Ejemplo en main.kv

```
Button:
    text: 'Mesero'
    font_size: 24
    size_hint_y: None
    height: 60
    on_press: app.record_event('Métrica 1: Botón Mesero'); root.manager.current = 'mesero'
```

- Registro de Envío de Pedido (Lógica): El registro se realiza en el método de Python `MeseroScreen.enviar_pedido()`, ya que este evento está ligado a una validación y el paso de datos (mesa, pedido). Esto permite que la métrica de Enviar Pedido incluya los datos relevantes (`{'mesa': mesa, 'pedido': pedido}`) en el campo `details` del JSON.

3.4. Gestión de Archivos y Persistencia

- `FileHandler`: Se configuró un `logging.FileHandler` que apunta a `metrics.log`. Este *handler* está configurado con un `Formatter('%(message)s')`, asegurando que cada línea en el archivo sea una entrada JSON limpia y completa, sin *metadata* de *logging* adicional.
- *Persistencia*: La centralización en el `FileHandler` garantiza que todos los eventos registrados a través de `record_event` se escriban en el archivo de forma automática y asíncrona, cumpliendo con el requisito de generación de archivos de log.



PlatoListo

Plato Listo

¿Cómo quieres entrar?

Mesero

Chef

Mesero - Toma de Pedido

7

hamburguesa sin pepinillos

Enviar pedido

Volver al menú principal

PlatoListo

Chef - Pedidos recibidos

Mesa 7: hamburguesa sin pepinillos

Volver al menú principal

PlatoListo

Plato Listo

¿Cómo quieres entrar?

Mesero

Chef

```
33 {"timestamp": "2025-12-06T00:01:45.382704Z", "event": "Métrica 1: Inicio App", "details": {"ts": "2025-12-06T00:01:45.382704Z"}}
34 {"timestamp": "2025-12-06T00:03:18.794680Z", "event": "Métrica 1: Botón Mesero"}
35 {"timestamp": "2025-12-06T00:04:19.718899Z", "event": "Métrica 1: Enviar Pedido", "details": {"mesa": "7", "pedido": "hamburguesa sin pepinillos "}}
36 {"timestamp": "2025-12-06T00:04:43.155474Z", "event": "Métrica 1: Botón Chef"}
37 {"timestamp": "2025-12-06T00:05:52.668736Z", "event": "Métrica 1: Duración Sesión", "details": {"start_ts": "2025-12-06T00:01:45.382704Z", "end_ts": "2025-12-06T00:05:52.668736Z", "duration_secon
```

4.Resultados de la Implementación de Métricas

La instrumentación del código mediante el sistema `record_event` y el `FileHandler` de logging demostró ser efectiva y producir logs estructurados.

Generación y Persistencia del Archivo el archivo `metrics.log` se generó automáticamente en la ruta configurada (junto al script principal), persistiendo los datos registrados a través de múltiples ejecuciones de la aplicación.

Datos Obtenidos

Se realizó una sesión de prueba para validar el registro de las métricas. El análisis del archivo `metrics.log` confirmó que cada entrada es un objeto JSON válido y contiene los datos requeridos:

Registro de Ciclo de Vida: Se capturó con éxito el evento Métrica 1: Inicio App al arranque y el evento Métrica 1: Duración Sesión al cierre.

Ejemplo de Resultado: Tras una sesión de prueba el log mostró la clave `duration_seconds`, validando el cálculo preciso del tiempo de sesión.

Registro de Interacción: El evento clave Métrica 1: Enviar Pedido fue registrado exitosamente, y lo más importante, incluyó los detalles funcionales del negocio en la clave `details`.

5. Reflexión sobre el uso de Inteligencia Artificial

La inteligencia artificial (IA) se ha convertido en uno de los temas más relevantes en el último tiempo. Con la rápida evolución de tecnologías que permiten a las máquinas aprender, adaptarse y tomar decisiones, la IA plantea tanto oportunidades sin precedentes como desafíos significativos para la humanidad.

El uso de la IA en este proyecto fue esencialmente técnico, enfocándose en la optimización de buenas prácticas. Su valor principal fue actuar como un asistente para la instrumentación de métricas y guía para resolver problemas.

Prompts utilizados

1¿Cuál es la mejor manera de registrar métricas estructuradas?

La mejor manera es utilizando el módulo **logging** de Python en conjunto con el módulo **json**.

- **Técnica:** Crear una función central (**record_event**) que reciba los datos, los **serialice a formato JSON** (mediante **json.dumps**), y luego use el **logger** para escribir esa cadena JSON en el archivo (**metrics.log**).
- **Ventaja:** Esto asegura que cada línea en tu archivo de log sea un **objeto estructurado** y estandarizado, facilitando su análisis posterior por *scripts* o herramientas externas.

2¿ cómo podría empezar hacer este proyecto?

Debes comenzar por la parte del **código y las métricas**, ya que la paquetización es el paso final.

1. **Código Base:** Define o elige tu aplicación Kivy funcional mínima (Ej: el contador o la app Plato Listo con sus pantallas).
2. **Configurar Logs:** En el archivo principal (**main.py**), implementa la función central **record_event** y configura el **FileHandler** para crear el archivo **metrics.log**.
3. **Integrar Métricas:** Coloca las llamadas a **record_event** en las partes clave del código (en **on_start()**, **on_stop()**, y en los *callbacks* de los botones en el archivo **.kv**).
4. **Repositorio:** Inicializa tu **repositorio GitHub** y comienza a subir tus *commits* de forma coherente.

3¿Cómo hacer la apk con Buildozer?(no entendi como hacerlo con linux)

La compilación a APK se realiza completamente en un entorno Linux (como **WSL/Ubuntu**) y requiere Buildozer para automatizar el proceso.

Pasos Breves:

1. **Preparar el Entorno:** Instala **Buildozer** y dependencias (Cython, OpenJDK 17) en tu terminal Linux.
2. **Inicializar:** Navega a la carpeta de tu proyecto Kivy y ejecuta **buildozer init**. Esto crea el archivo de configuración **buildozer.spec**.
3. **Configurar:** Edita **buildozer.spec** para definir el nombre de tu app, dominio, y, crucialmente, la lista de requisitos (**requirements: python3,kivy,android**).
4. **Compilar:** Ejecuta el comando **buildozer -v android debug**. La primera vez, Buildozer descargará las herramientas de Android (SDK, NDK); las veces siguientes será más rápido.
5. **Resultado:** El archivo **.apk** se encontrará en la carpeta **/bin/** de tu proyecto.

4¿qué es la implementación de métricas?

La implementación de métricas es el proceso de **instrumentar** (agregar código de medición) una aplicación para **registrar, cuantificar y analizar** el comportamiento del usuario y el rendimiento de la aplicación.

- **En tu Proyecto:** Consiste en añadir las llamadas a la función **record_event** para capturar cuándo inicia la app, cuánto tiempo está activa, y qué botones se presionan, guardando esta información en archivos **.log** para su posterior análisis.
- **Objetivo:** Transforma el uso de la aplicación en datos objetivos que permiten tomar decisiones informadas sobre la usabilidad y la mejora de la aplicación.

Ajustes realizados.

6. Ajustes realizados

Los siguientes ajustes fueron implementados sobre las sugerencias iniciales de la IA para garantizar la funcionalidad, la portabilidad y la conformidad con los estándares de buenas prácticas:

Uso de Rutas Relativas al Script Ajuste: Se utilizó la construcción `os.path.join(os.path.dirname(__file__), 'metrics.log')` para definir la ruta del archivo de log, en lugar de una ruta relativa simple (`metrics.log`)

Propósito Asegurar que el archivo de log se cree siempre en la ubicación esperada (junto al *script* principal), sin importar el directorio de trabajo actual desde donde se ejecute la aplicación.

Uniformidad del Timestamp (UTC) ajuste se modificó la generación del *timestamp* para utilizar la función `datetime.datetime.utcnow()` en lugar de `datetime.now()`, añadiendo el sufijo 'Z' al formato ISO 8601.

Propósito: asegurar que todos los registros de métricas estén en Hora Universal Coordinada (UTC). Esto elimina la ambigüedad de zona horaria, lo cual es fundamental para el análisis posterior de cualquier aplicación.



7.conclusion

La fase de desarrollo de este proyecto termina con la implementación de un sistema de métricas y registro de eventos, lo cual valida el dominio de las buenas prácticas de instrumentación en el desarrollo de aplicaciones con Python y Kivy.

Aunque la fase de paquetización con Buildozer no fue abordada en este informe, el proyecto sienta una base sólida: la aplicación es completamente instrumentada y lista para la fase final de despliegue, donde sus métricas podrán ser capturadas en el entorno móvil de Android y dar las gracias al profesor Cristian ya que durante todo este año fue un profesor comprensivo y motivado por ayudar a sus estudiante yo le agradezco mucho este año y por la oportunidad de entregar este y otros más trabajos fuera del plazo dado.